MATH300: Mathematical Computing · Fall 2026

Project: Finding Shortest Paths, Old and New

Groups must be formed, and project titles submitted on Canvas, by **October 26, 2026**. The project has two graded deliverables and an oral exam: a 3–5 page report for Project 2 (due **November 20, 2026**), a poster for the final project (due **December 11, 2026**), and an oral exam held between November 30 and December 17, 2026, 3:30 pm. Read the general rules below before you start; they apply throughout the project.

General remarks

- Projects are set for a group of **two or three** students.
- Form your group by **October 26, 2026**, and submit your project title as a group assignment on **Canvas** by the same date.
- Every group member must carry an **equal portion** of the coding and of the report and poster preparation.
- The scope can be adjusted to the number of participants, their interests and prior knowledge.
- Only **Python** may be used for programming.
- Implementation, documentation, validation and demonstration examples are all part of the project.
- Within these rules you are free to choose the implementation, code structure and exact content. You can discuss this with me at any time.

Version control and GitHub

Each group keeps **one GitHub repository for the whole project** (Project 2 and the final project), and every member commits from their own GitHub account.

- Use git from the very beginning. Commits should be small, consistent and clearly described. Avoid large commits.
- Each group member uploads their own code using **their own GitHub account**. Commits from a shared or borrowed account will not be credited to the other members.
- The repository name is up to your group.
- If the repository is private, add my GitHub account `say-yas` as a collaborator.
- The repository must contain a README describing the project, how to install and run the code, and the AI-use statement (see below).

Use of AI tools

AI-generated code is not accepted in any form. Code produced by a chat assistant or code generator (e.g. ChatGPT, Claude, Gemini) and copied, pasted or adapted into your project will be treated as work that is not your own.

- **Permitted:** inline coding assistance such as GitHub Copilot autocompletion while you write the code yourself, **provided its use is acknowledged**.
- **Acknowledgement:** include an AI-use statement in the README and in the report that names the tool(s) used and the parts of the code (files or functions) where they assisted. If no AI tools were used, state this explicitly.
- **Reports and posters must not be generated by AI** in any form: their text, figures and layout must be your own work. If AI tools were used for any other non-code purpose (e.g. checking grammar or searching the literature), describe this in the same statement.
- You are responsible for every line in your repository. In the oral exam you must be able to explain and modify any part of the code you submitted.

Deliverables and deadlines

Submit each deliverable following the details provided on the **Canvas** page. The written report and poster are handed in on **Crowdmark**, and the link to your GitHub repository is submitted separately on **Canvas**.

Deliverable	**Due**	**Submission**	**Notes**
Group and project title	Monday, October 26, 2026	Group assignment on Canvas	Group of two or three members; submit the project title.
Project 2 report	Friday, November 20, 2026	Report (PDF) on Crowdmark; repository link on Canvas	3–5 pages; the page limit **includes** references, figures and appendices.
Final project poster	Friday, December 11, 2026	Poster (PDF) on Crowdmark; repository link on Canvas	36 inches high by 48 inches wide (landscape). Target audience: undergraduate classmates with a basic background in math, physics or engineering.
Oral exam	Between November 30 and December 17, 2026, 3:30 pm	Individual meeting; details discussed in class	30 minutes per student. Your poster must already be submitted on Crowdmark before your exam. The last possible slot ends December 17 at 3:30 pm.

Oral exam

The oral exam is individual and lasts 30 minutes per student. It takes place between November 30 and December 17, 2026, 3:30 pm. The oral exams and how to arrange them will be discussed in class before the Thanksgiving break.

Before your oral exam, your poster must also be submitted on **Crowdmark**; that is, you must have finished both Project 2 and the final project. In the meeting you will:

- Present your results and algorithms.
- Possibly live-code and run parts of your algorithms to generate outputs on the spot.
- Answer questions about the theory, your implementation and your results. Each member should be able to explain the whole project, not only their own parts.

Bring a laptop with the repository cloned and a working environment, so that the code runs without additional setup.

Grading

Component	Points
Group and project title (on time)	10
Project 2 report	100
Final project poster	100
Oral exam (individual)	50
Bonus: extra steps beyond the required tasks	up to 10

The Project 2 report and the poster are each graded out of 100. What matters most is:

- **Coherence:** a clear story from the question, through the methods, to the results and conclusions.
- **Correctness:** of the mathematics, the code and the results.
- **Depth of analysis and discussion:** explain what your results mean, why they look the way they do, and what their limitations are, rather than only showing them.
- **Professional presentation:** figures with labeled axes and readable font sizes, captions and legends, a clean and readable layout, and proper references.

A complete, correct and well-discussed solution of the required tasks can earn full marks. Tasks marked **(Bonus)** and the suggestions under "Going further" are optional; exploring such extra steps can earn **up to 10 bonus points** in total.

Requirements for the report and the poster

Both the report and the poster must contain a clear description of the project, references, properly labeled figures and an author-contributions statement.

Content

- A clear and concise description of the problem, the methods and the results.
- The AI-use statement described above.
- The report should not contain any significant amount of code; refer to the repository instead.

Poster

- The poster must be **self-contained**: a reader should be able to follow it without the report, the code or a spoken explanation.
- You may be selective in what you show and how you narrate your story, so that an undergraduate audience with a basic background in math, physics or engineering can follow it.

- You may choose your own title for the poster; be creative.

References

- Include a list of references at the end of the report and the poster.
- Cite sources clearly where they are used in the text (papers, books, software, code repositories and data sets).
- Use one consistent citation style throughout.

Figures

- Every plot must have labeled x and y axes (and z axis for 3D plots or color bars), including units where applicable.
- Axis labels, tick labels and legends must have a font size similar to the surrounding text, so they remain readable without zooming.
- Each figure needs a legend when it shows more than one data series, and a caption that states what is shown.
- Use log scales where appropriate (e.g. for convergence plots) and say so in the caption.

Author contributions

Each group must include an author-contributions statement in the report and the README that states:

- Who did what: which parts of the code, experiments, analysis, writing and poster each member worked on.
- To what extent: the approximate share of each member in each part (e.g. 50 % / 50 %).
- Who took the lead in which parts.

The coding and the report and poster preparation must be shared equally between the members. The commit history should be consistent with the stated contributions.

Codebase requirements

- Well structured and documented, both in the code (docstrings, comments) and in the repository (README).
- Validated: show that your implementation produces correct results, e.g. on examples with known solutions.

Project description

Goal

How does a navigation app find the fastest route? This is the *shortest path problem*. In this project you will implement and compare two algorithms for it in Python:

- **Dijkstra's algorithm** [1], the classical method from 1959, which is still used everywhere today;
- a **new algorithm** from 2025 by Duan et al. [2], which is the first to be provably faster than Dijkstra's algorithm on large sparse graphs.

You do **not** need to parallelize your code.

Background

A *graph* is a set of points (vertices) connected by arrows (edges). Each edge has a length, called its *weight*, which is never negative; think of cities connected by one-way roads with travel times. Given a starting vertex s, we want the length of the shortest route from s to every other vertex. We write n for the number of vertices and m for the number of edges.

Dijkstra's algorithm works like a wave spreading out from s. It keeps a tentative distance for every vertex. At each step it picks the closest vertex not yet finished, marks it as finished, and checks whether going through it gives a shorter route to its neighbors. A priority queue (a heap) finds the closest vertex quickly, and the total running time is about $m \log n$.

The new algorithm [2] starts from a simple observation: Dijkstra's algorithm finishes the vertices in order of their distance from s, so it is effectively *sorting* them, and sorting has a cost that cannot be avoided. The new algorithm avoids sorting all the vertices. It splits the problem into smaller pieces by distance, handles each piece recursively, and uses a few rounds of simple edge-by-edge updates (as in the Bellman–Ford algorithm) to deal with vertices whose distance is easy to find. In theory this is faster than Dijkstra's algorithm for very large graphs. The paper gives the algorithm as pseudocode, which you can follow step by step.

Tasks for Project 2 (report, due November 20, 2026)

1. Store a graph in Python, for example as a dictionary of adjacency lists, and write a function that creates random graphs with a given number of vertices and edges and random non-negative weights.
2. Implement Dijkstra's algorithm using Python's `heapq` module [3], and return the distances.
3. Implement the new algorithm from [2] by following its pseudocode. You may simplify it: for example, you may use a simple sorted list or a heap instead of the special data structure in the paper, and you may skip the step that changes the graph so that every vertex has at most two incoming and two outgoing edges. Say clearly what you simplified.
4. Check both implementations on small graphs that you can solve by hand, and check that they give the same distances on larger random graphs.
5. **(Bonus)** Also return the shortest route to each vertex, not only its distance.
6. **(Bonus)** Implement the special data structure from the paper instead of a simple sorted list or heap.

Tasks for the final project (poster, due December 11, 2026)

1. Compare the running times of the two algorithms on graphs of increasing size and plot the results.

2. Discuss what you observe: which algorithm is faster for the graphs you tried, and why might the new algorithm not be faster in practice, even though it is faster in theory? What are the benefits and shortcomings of each algorithm?
3. **(Bonus)** Count basic operations, such as the number of edge updates, for both algorithms and compare them.

Going further

These optional steps count toward the bonus points. For example, you can test the algorithms on grid graphs or a small real road map.

References

[1] E. W. Dijkstra (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271. https://doi.org/10.1007/BF01386390

[2] R. Duan, J. Mao, X. Mao, X. Shu and L. Yin (2025). Breaking the sorting barrier for directed single-source shortest paths. arXiv:2504.17033. https://arxiv.org/abs/2504.17033

[3] Python Software Foundation. *heapq — Heap queue algorithm*, Python documentation. https://docs.python.org/3/library/heapq.html